Jornada de Ensino, Ciência, Extensão e Tecnologia do IFSP – Campus Jacareí

JECET 2026 · INOVAÇÃO · IFSP CAMPUS JACAREÍ

FotoFácil: um editor de imagens por nós para automatizar edições em lote *sem programar*

Ferramenta de código aberto em que o processamento de imagens é montado por blocos: você desenha o fluxo na tela, o sistema calcula a ordem de execução e aplica a mesma receita em milhares de arquivos.

processamento de imagens · programação visual · automação em lote · código aberto

Nathan Henrique Guimarães de Oliveira
IFSP · guimaraes.nathan@aluno.ifsp.edu.br

Ana Paula Abrantes de Castro Shiguemori
IFSP · anapaula.acs@ifsp.edu.br

Uma imagem digital é uma matriz de números

Cada valor da matriz é a intensidade de um ponto. Processar imagem é aplicar contas sobre essa matriz: somar brilho, comparar vizinhos, filtrar frequências.

- **Cotidiano:** ajustar uma foto, melhorar um documento escaneado, tratar a imagem de uma câmera.
- **Pesquisa:** preparar, padronizar e ampliar milhares de imagens antes de treinar um modelo.
- **O ponto comum:** o ajuste é sempre um **algoritmo**. Feito à mão, ele se repete e varia; descrito como fluxo, ele se reproduz.

1.100×785
A FOTO USADA NESTA APRESENTAÇÃO

860 mil
VALORES DE INTENSIDADE EM UMA SÓ IMAGEM

95,3%
DOS PIXELS MUDAM DE VALOR AO EQUALIZAR



Imagem de referência do trabalho. Todas as figuras desta apresentação saem dos mesmos nós do FotoFácil, aplicados sobre ela.



A imagem por dentro. Recorte ampliado com os números que o programa realmente lê.

Automatizar em lote hoje exige programar

O QUE JÁ EXISTE

Editores gráficos e ferramentas visuais

— **GIMP e Photoshop:** fazem lote, mas por uma linguagem de script própria: a automação só chega para quem programa.

— **n8n:** automação visual comprovada, com nós genéricos de dados, sem processamento de imagem nativo.

— **Orange:** programação visual de mineração de dados, com foco em tabelas e modelos, não em pixels.

O QUE ISSO CUSTA

Quem não programa faz no braço

— Abrir, ajustar, exportar. Repetir para cada arquivo.

— O critério muda no meio do caminho e o lote deixa de ser comparável.

— Na pesquisa, o esforço de preparar o dado compete com o próprio método.

A lacuna: automação visual, que já provou funcionar em outras áreas, não chegou ao processamento de imagens.

A PROPOSTA

Montar o processamento como um **fluxo de blocos conectados**: cada bloco é uma operação conhecida de processamento digital de imagens e a ligação entre eles define a ordem. O mesmo desenho roda em uma imagem ou em milhares.

O que o trabalho se propôs a fazer

OBJETIVO GERAL

Desenvolver um editor de imagens de código aberto em que o processamento é montado visualmente por nós, permitindo aplicar a mesma receita em lotes de imagens sem escrever código.

OBJETIVO 1

Montar sem código

Interface de arrastar e soltar que dispensa conhecimento de programação.

OBJETIVO 2

Repetir em lote

Executar o mesmo fluxo sobre muitos arquivos, com resultado previsível.

OBJETIVO 3

Cobrir PDI clássico

Do ajuste básico à análise: cor, filtros, bordas, frequência.

OBJETIVO 4

Servir à pesquisa

Preparar conjuntos de imagens para treinar modelos de IA.

PALAVRAS-CHAVE DO RESUMO APROVADO

processamento de imagens

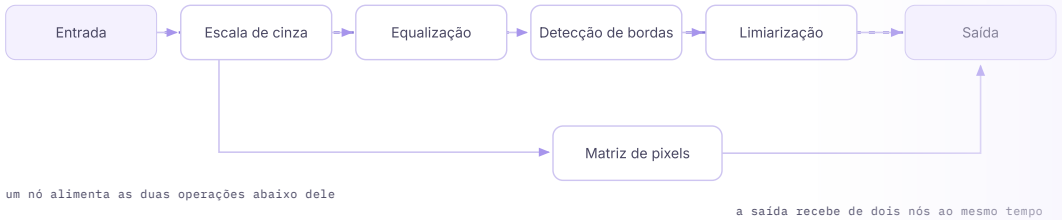
programação visual

automação em lote

arquitetura de software

O desenho é a receita; o sistema calcula a ordem

Cada nó declara de quais nós depende. Com isso o backend monta um **grafo acíclico dirigido**, resolve a ordem com uma ordenação topológica e executa cada operação uma única vez, reaproveitando resultados que duas ramificações compartilham.



REGRA 1

Cada operação roda **uma vez**, mesmo que dois caminhos precisem dela.

REGRA 2

A ordem sai do próprio desenho: quem depende vai depois.

REGRA 3

Um ciclo (A depende de B que depende de A) é recusado antes de executar.

Duas camadas, conversando em tempo real

CAMADA 1 · INTERFACE

Electron + React + TypeScript

arrastar e soltar, painel de parâmetros, prévia do resultado

WebSocket

CAMADA 2 · NÚCLEO

Go

grafo, ordenação, execução do lote, persistência em SQLite

SEM NUVEM

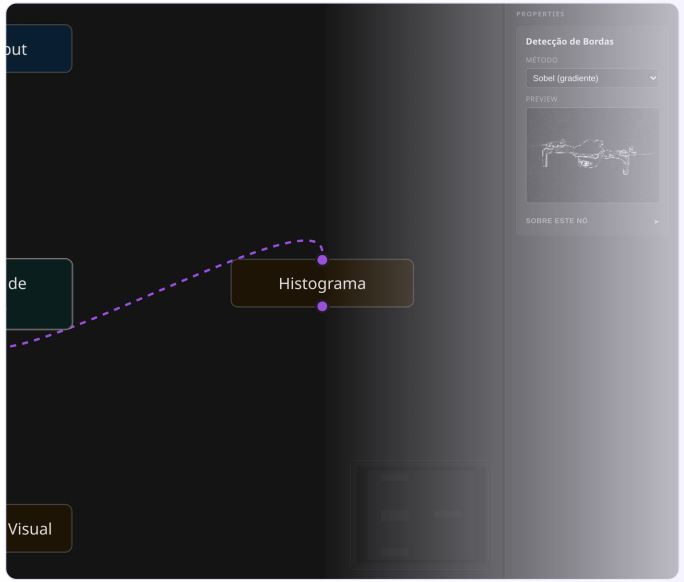
Roda na máquina do usuário. Nenhuma imagem sai do computador.

PRÉVIA VIVA

Cada mudança no fluxo devolve uma imagem de conferência.

FLUXOS SALVOS

O desenho fica guardado e pode ser reaberto.



A ferramenta em uso. O canvas com o grafo montado à esquerda e, à direita, os parâmetros do nó selecionado com a prévia do resultado.

28 tipos de nó, em 8 categorias

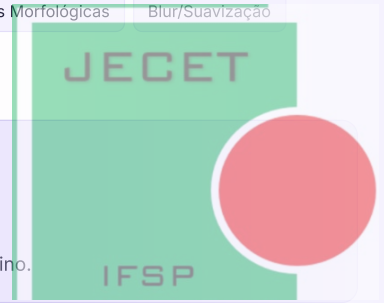
- Filtros & Realce 8
- Sensoriamento Remoto 6
- Visualização & Análise 4
- Dataset / ML 3
- Entrada/Saída 2
- Ajustes Básicos 2
- Transformação 2
- Inteligência Artificial 1

FILTROS & REALCE · 8 NÓS · CLIQUE NOUTRA CATEGORIA PARA VER OS NÓS DELA

Convolução | Equalização de Histograma | Detecção de Bordas | Operações Morfológicas | Blur/Suavização | Ruído | Limiarização | FFT

TRÊS FRENTES DENTRO DO CATÁLOGO

- **Edição:** recorte, rotação, brilho, contraste.
- **PDI clássico:** cor, suavização, bordas, limiarização, frequência.
- **Preparação de dados:** augmentação e exportação rotulada para treino.



Antes de codar, o resultado esperado

DESENVOLVIMENTO ORIENTADO A TESTES

Para cada operação o teste vem primeiro: ele monta uma matriz conhecida, aplica o nó e compara **pixel a pixel** com o resultado esperado. Só então a implementação é escrita, até o teste passar.

- Erro de arredondamento, borda ou canal aparece na hora.
- Mudar um nó sem quebrar outro deixa de depender de memória.
- É o que permite afirmar que os resultados desta apresentação são os do algoritmo.

94

TESTES AUTOMATIZADOS NO NÚCLEO

28

NÓS COBERTOS POR VERIFICAÇÃO

DOIS TESTES REAIS DO PROJETO: ESCOLHA UM E VEJA O CÓDIGO

média em imagem uniforme | mediana contra o ruído sal

```
func TestBlurNodeMeanOnUniformImageStaysUniform(t *testing.T)
{
    img := solidColorImage(5, 5, 100, 100, 100)
    ctx := &ProcessContext{Images: []image.Image{img}}
    node := &BlurNode{ID: "blur1", Method: "mean",
KernelSize: 3}

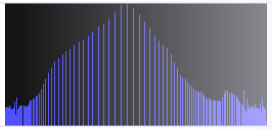
    if err := node.Process(ctx); err != nil {
        t.Fatalf("BlurNode failed: %v", err)
    }
    r, _, _, _ := ctx.Images[0].At(2, 2).RGBA()
    if r>>8 != 100 { // o miolo tem que continuar 100
        t.Errorf("expected uniform image to remain uniform
after mean blur, got %d", r>>8)
    }
}
```

O teste descreve o resultado esperado; a implementação vem depois, até passar. Foi assim para os 28 nós.

Escala de cinza e equalização



arraste sobre a imagem para comparar · resultado produzido pelo nó de equalização do FotoFácil



Histograma: quantos pixels existem em cada nível de intensidade. A equalização redistribui essa distribuição.

MEDIDA NA IMAGEM EQUALIZADA	VALOR
Intensidade média	128,44
Mediana	131,00
Desvio padrão (contraste)	73,40
MSE / PSNR contra a escala de cinza	1950,91 / 15,23 dB

O desvio padrão mede o contraste: quanto maior, mais a imagem usa os extremos. É esse número que muda quando a equalização funciona.

Encontrar contornos e separar o que é objeto



Ponto de partida: a luminância de cada pixel, calculada pelo nó de espaço de cor.

escala de cinza | detecção de bordas (Sobel) | Laplaciano | limiarização (Otsu) | dilatação 5×5

O QUE ESTÁ POR TRÁS DOS BOTÕES

- Bordas não são contorno desenhado: são a **variação** entre vizinhos.
- A limiarização decide o que é objeto usando o próprio histograma.
- A morfologia corrige o que a limiarização deixou quebrado.

LEITURA DOS NÚMEROS

Sobel deixa **75%** da área praticamente preta e apenas **5,3%** dela passa da intensidade 25. Otsu deixa **22,5%** em branco e a dilatação 5×5 eleva para **25,0%**. Tudo medido na saída dos próprios nós.

Da pasta de fotos ao conjunto de treino



entrada



rotação



espelhamento



outra variação

O nó de **augmentação** gera variantes da mesma imagem para ampliar o conjunto. Ele usa uma **semente fixa**: quem repetir o fluxo obtém exatamente as mesmas variantes. É a diferença entre dados aumentados e dados aumentados de forma reprodutível.



com o nó em 6% de sal e pimenta: o desvio padrão sobe de 37,5 para 43,3



depois da mediana 3×3: o desvio volta a 37,3, sem borrar a imagem

CONJUNTO EXPORTADO PELO NÓ DE SAÍDA ROTULADA

```
saida/
├─ train/
│  ├─ exp_0.png
│  ├─ exp_1.png
│  └─ exp_2.png
├─ val/
│  └─ exp_3.png
└─ test/
   ├─ exp_4.png
   └─ exp_5.png
```

6 imagens: 60% treino, 20% validação e o resto em teste, já em pastas que um framework de treino lê de imediato.

semente 42 · resultado repetível

POR QUE ISSO FECHA O CICLO

O gargalo de quem pesquisa era preparar o dado; aqui ele virou um nó. A mesma ferramenta que ajusta a foto organiza o conjunto de treino.

Objetivo por objetivo, e o que ainda falta

OBJETIVOS RESPONDIDOS

- ✓ **Montar sem código:** os nós são ligados por arrastar e soltar e a ordem é calculada pelo sistema.
- ✓ **Repetir em lote:** atingido com ressalva, a execução ainda é **sequencial**: o ganho vem da repetição, não do paralelismo.
- ✓ **PDI clássico:** 28 nós, de brilho e contraste a Fourier, limiarização e morfologia, todos verificados por teste.
- ✓ **Servir à pesquisa:** o conjunto exportado sai dividido em treino, validação e teste.

LIMITES CONHECIDOS

- — **O fluxo não é exportável:** o desenho fica salvo na máquina, mas ainda não sai como arquivo para outra pessoa abrir.
- — **A IA do catálogo é heurística:** conta e mede área de objeto, não é um modelo treinado.
- — **Nós de sensoriamento remoto:** dependem de imagens com bandas específicas, ainda não validados sobre dados reais de satélite.
- — **Persistência:** encontramos um caso em que o fluxo aberto pode ser gravado vazio antes de ser lido do banco; está registrado e é o primeiro item da fila.

PRÓXIMOS PASSOS

A RESPOSTA À PERGUNTA DO INÍCIO

Sim: montando o processamento como fluxo, dá para aplicar a mesma edição a milhares de imagens sem programar, e repetir exatamente o mesmo resultado depois.

Bases do trabalho e onde encontrá-lo

— **GONZALEZ, R. C.; WOODS, R. E.** *Digital Image Processing*. 4. ed. Pearson, 2018.

— **OTSU, N.** A threshold selection method from gray-level histograms. *IEEE Trans. on Systems, Man, and Cybernetics*, v. 9, n. 1, p. 62-66, 1979.

— **COOLEY, J. W.; TUKEY, J. W.** An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, v. 19, n. 90, p. 297-301, 1965.

— **SOUSA, T.** Dataflow programming: concept, languages and applications. 2012.

— **DEMŠAR, J. et al.** Orange: data mining toolbox in Python. *JMLR*, v. 14, p. 2349-2353, 2013.

— **BECK, K.** *Test-Driven Development: By Example*. Addison-Wesley, 2002.

— **GIMP Development Team** e **n8n**: documentação oficial de automação em lote e de fluxos visuais.

— Documentação técnica do projeto: referência dos 28 nós, mapa da arquitetura e guia de execução.

OBRIGADO · PERGUNTAS?

AUTOR
Nathan Henrique Guimarães de Oliveira

ORIENTAÇÃO
Ana Paula Abrantes de Castro Shiguemori

CONTATO
guimaraes.nathan@aluno.ifsp.edu.br

REPOSITÓRIO DO PROJETO
github.com/nathanhgo/foto-facil